



NATIONAL INSTITUTE OF TECHNOLOGY SILCHAR

Department of Computer Science & Engineering

PERCEPTUAL IMAGE HASHING USING CONVOLUTIONAL NEURAL NETWORKS

A Project Report Submitted in Partial Fulfilment of the
Requirements for the 8th Semester B.Tech. Project

by

Mst. Habiba Akter Bithi (2012162)

Parijat Biswas Pritom (2012164)

Muhammad Shakhawat Hossain (2012172)

Under the Supervision of

Mr. Prabhakar Sarma Neog

**COMPUTER SCIENCE & ENGINEERING DEPARTMENT
NATIONAL INSTITUTE OF TECHNOLOGY SILCHAR**

May, 2024

© NATIONAL INSTITUTE OF TECHNOLOGY SILCHAR, MAY 2024
ALL RIGHTS RESERVED

Certificate

Department of Computer Science & Engineering

NATIONAL INSTITUTE OF TECHNOLOGY SILCHAR

It is certified that the work contained in this thesis entitled “**Perceptual Image Hashing Using Convolutional Neural Networks**” submitted by **Mst. Habiba Akter Bithi (2012162), Parijat Biswas Pritom (2012164), and Muhammad Shakhawat Hossain (2012172)** for the B.Tech. End Semester Project Examination, May 2024, is a bona fide record of the work carried out by them under my supervision.

Place: NIT Silchar

Date: 15/05/2024

Mr. Prabhakar Sarma Neog
Computer Science & Engineering
National Institute of Technology Silchar

Declaration

Department of Computer Science & Engineering

NATIONAL INSTITUTE OF TECHNOLOGY SILCHAR

Thesis Title: Perceptual Image Hashing Using Convolutional Neural Networks

Degree for which the Thesis is submitted: Bachelor of Technology

We hereby declare that the work presented in this thesis represents, to the best of our knowledge, our own ideas and effort expressed in our own words. Wherever the ideas or words of others have been used, they have been duly acknowledged and cited in the list of references. This thesis has been prepared without resorting to plagiarism, and we have adhered to the principles of academic honesty and integrity throughout. No falsified or fabricated data have been presented in this work. We understand that any violation of the above may attract disciplinary action from the Institute, including revocation of the conferred degree, and may also invite penal action from the sources that have not been properly cited or from whom due permission has not been obtained.

Signed with Date:

Mst. Habiba Akter Bithi
Parijat Biswas Pritom
Muhammad Shakhawat Hossain

“You have to dream before your dreams can come true.”

— A. P. J. Abdul Kalam

Abstract

Perceptual image hashing is a technique for generating compact binary representations, or hashes, of digital images. Unlike cryptographic hashes, which change drastically with even a single-bit modification of the input, perceptual hashes are designed to remain stable under common content-preserving distortions such as compression, noise addition, filtering, and geometric transformations, while still being sensitive to genuine changes in perceptual content. This property makes perceptual hashing valuable for applications including image authentication, copyright and copy detection, near-duplicate image retrieval, and forensic analysis.

In recent years, convolutional neural networks (CNNs) have emerged as a powerful tool for automated image feature extraction, learning hierarchical representations that are naturally robust to many of the distortions that challenge traditional, hand-crafted hashing schemes. This project investigates a CNN-based perceptual image hashing scheme built around pretrained deep architectures — principally VGG16, along with VGG19 and AlexNet for comparison. The proposed pipeline consists of three stages: pre-processing, in which input images are normalised to a standard size and format; feature extraction, in which convolutional layers of a pretrained network produce a high-dimensional feature vector for each image; and hash generation, in which the feature vector is aggregated, quantised, and discretised into a fixed-length binary hash code.

To evaluate the scheme, an experimental dataset was constructed from 50 representative images, each subjected to ten categories of content-preserving transformations — including rotation, speckle noise, Gaussian and median filtering, gamma correction, and adjustments to brightness, colour, and sharpness — at ten parameter settings each, yielding 5,000 similar image pairs and 245,000 dissimilar image pairs. Hamming distances between the hash codes of original and transformed images were computed and compared against ground-truth similarity labels across a range of decision thresholds. The resulting Receiver Operating Characteristic (ROC) curve demonstrates that the VGG16-based hashing scheme achieves a high true positive rate at a very low false positive rate, indicating strong robustness to content-preserving distortions together with good discriminative power between dissimilar images. These results suggest that CNN-based perceptual hashing is a promising and practical approach for real-world image authentication and retrieval applications.

Acknowledgements

We take this opportunity to express our sincere gratitude and heartfelt thanks to our supervisor, Mr. Prabhakar Sarma Neog, Department of Computer Science and Engineering, National Institute of Technology Silchar, for his continuous inspiration, patient guidance, and valuable feedback at every stage of this project. His expertise, encouragement, and willingness to help us navigate the intricacies of this work have played a pivotal role in shaping both the project and our own understanding of the subject.

We would also like to thank all the faculty members of the Department of Computer Science and Engineering, National Institute of Technology Silchar, for their support and encouragement throughout the various phases of this work. Their contributions, direct and indirect, have been indispensable to the completion of this project, and we remain sincerely grateful for their efforts.

Mst. Habiba Akter Bithi
Parijat Biswas Pritom
Muhammad Shakhawat Hossain

Contents

Certificate.....	iii
Declaration.....	iv
Abstract.....	vi
Acknowledgements.....	vii
List of Figures.....	ix
List of Tables.....	ix
Chapter 1 Introduction.....	1
1.1 Motivation.....	1
1.2 Problem Statement.....	2
1.3 Objectives.....	2
1.4 Organisation of the Thesis.....	3
Chapter 2 Literature Survey.....	4
Chapter 3 Proposed System.....	6
3.1 Mathematical Preliminaries of Convolutional Neural Networks.....	6
3.2 Convolutional Architectures Considered.....	8
3.3 Formal Hash Generation and Similarity Measurement.....	10
3.4 Dataset.....	12
3.5 Image Transformations.....	13
3.6 Ground Truth Preparation and Evaluation Framework.....	14
Chapter 4 Experimental Results and Discussion.....	17
4.1 Evaluation Metrics.....	17
4.2 ROC Curve Construction.....	17
4.3 Results.....	18
4.4 Discussion.....	19
Chapter 5 Conclusion and Future Work.....	21
References.....	22

List of Figures

Figure 3.1	VGG-16 convolutional network architecture.....
Figure 3.2	VGG-16 feature extraction workflow.....
Figure 3.3	Representative dataset samples.....
Figure 3.4	Structure of a confusion matrix.....
Figure 4.1	ROC curve for the VGG-16-based hashing scheme.....

List of Tables

Table 3.1	Image transformations and parameter values.....
Table 3.2	Summary of similar and dissimilar image pairs generated.....

List of Algorithms

Algorithm 1	CNN-Based Perceptual Hash Generation.....
Algorithm 2	Threshold Sweep and ROC Curve Construction.....

Chapter 1 Introduction

Perceptual image hashing is a technique for generating compact binary representations, or hash codes, of digital images. Unlike cryptographic hash functions, which are designed so that even a single-bit change in the input produces a completely different output, perceptual hash functions are designed to be robust to common, content-preserving image distortions — such as compression, additive noise, filtering, and mild geometric transformations — while still remaining sensitive to genuine changes in the visual content of an image. This dual requirement, robustness to benign modification alongside sensitivity to real content changes, is what distinguishes perceptual hashing from ordinary hashing and makes it especially useful for applications such as image authentication, copy detection, and content-based image retrieval.

In recent years, convolutional neural networks (CNNs) have emerged as a powerful and increasingly standard tool for image feature extraction. Through hierarchical layers of learned convolutional filters, CNNs are able to extract high-level features from images that are naturally invariant to many of the distortions that challenge traditional, hand-crafted hashing methods. This makes CNNs particularly well suited to the task of perceptual image hashing.

In this project, we propose and evaluate a perceptual image hashing scheme built on CNN-based feature extraction. The scheme consists of three main stages. In the pre-processing stage, input images are normalised to a standard size and format. In the feature extraction stage, a pretrained CNN — primarily VGG16, with VGG19 and AlexNet examined for comparison — is used to extract a feature vector from each image. In the hash generation stage, this feature vector is aggregated, quantised, and binarised to produce a fixed-length binary hash code. The resulting scheme is evaluated for its robustness to a range of content-preserving transformations and its ability to discriminate between genuinely different images.

1.1 Motivation

The motivation for using convolutional neural networks in perceptual image hashing arises from the limitations of traditional hashing techniques and the distinct advantages that deep learning approaches offer. The following points summarise the key motivations for this work.

1. Improved robustness to image transformations: Traditional perceptual hashing methods often struggle to remain robust against common image transformations such as cropping, scaling, rotation, and compression, which limits their effectiveness in practical settings. CNNs, in contrast, have demonstrated strong

robustness to such transformations, making them a more suitable foundation for perceptual hashing.

2. Enhanced discriminative capability: Accurately distinguishing between perceptually similar and dissimilar images is essential for applications such as image retrieval, authentication, and copyright protection. Traditional methods often fall short in this respect, producing false positives or false negatives. CNNs, owing to their ability to learn complex hierarchical features, can substantially improve discriminative power and reduce such errors.
3. Automated feature extraction: Traditional perceptual hashing methods typically rely on hand-crafted features, which require domain expertise and considerable manual effort to design. CNNs automatically learn relevant features directly from image data, removing the need for manual feature engineering.
4. Potential for end-to-end deep hashing: Deep hashing is an emerging paradigm that integrates hash-code generation directly into the CNN architecture, enabling joint optimisation of feature extraction and hash generation within a single trainable pipeline.

1.2 Problem Statement

Despite considerable progress, several persistent challenges remain in the design of effective perceptual image hashing schemes:

- Selecting transformations, invariant features, decompositions, or statistical properties of images that remain resilient against the widest possible range of attacks.
- For CNN-based algorithms, designing constrained loss functions — in addition to the network's primary loss — that penalise large hash distances between similar images while rewarding large hash distances between dissimilar images.
- Ensuring robustness against individual attacks as well as combinations of attacks applied simultaneously.
- Managing the inherent trade-off between robustness and discriminative capability, since a hash function optimised purely for robustness risks losing sensitivity to genuine content changes, and vice versa.

1.3 Objectives

This project is guided by the following objectives:

- To conduct a thorough survey of the existing literature on perceptual image hashing, with particular attention to CNN-based approaches.
- To design and implement a perceptual image hashing scheme that is resilient against a variety of content-preserving and content-altering attacks.
- To achieve a favourable balance between robustness and discriminative capability in the resulting hash function.
- To empirically evaluate the proposed scheme and assess its suitability for downstream applications such as image authentication and retrieval.

1.4 Organisation of the Thesis

The remainder of this thesis is organised as follows. Chapter 2 presents a survey of related work in perceptual image hashing, covering both classical and deep-learning-based approaches. Chapter 3 describes the proposed CNN-based hashing scheme in detail, including the underlying network architectures, the experimental dataset, the set of image transformations used to simulate content-preserving attacks, and the procedure used to prepare ground-truth labels. Chapter 4 presents the experimental results, including the evaluation methodology based on confusion matrices and ROC analysis, along with a discussion of the findings. Chapter 5 concludes the thesis and outlines directions for future work.

Chapter 2 Literature Survey

Perceptual image hashing has attracted sustained research interest over the past decade, with approaches ranging from classical statistical and transform-domain techniques to, more recently, deep-learning-based methods. This chapter surveys representative work in the area, with particular emphasis on CNN-based approaches that are most relevant to the scheme proposed in this thesis.

Li et al. [1] explored the use of convolutional neural networks for image hashing with an explicit focus on preserving perceptual similarity between related images, demonstrating that learned features could outperform hand-crafted descriptors on standard robustness benchmarks. Building on this line of work, Liu et al. [2] examined multi-scale CNN architectures for perceptual image hashing and showed that combining features extracted at multiple spatial resolutions improves robustness to a wider range of distortions than single-scale architectures alone.

A related direction incorporates perceptual cues more directly into the hashing pipeline. Xiaopeng Li et al. [3] proposed a deep perceptual hashing framework that embeds perceptual features within a CNN-based architecture, while a broader survey by the same lead author [4] catalogued the range of deep learning techniques — including CNNs, autoencoders, and hybrid architectures — that have been applied to image hashing more generally. Huh et al. [5] investigated the use of capsule networks, a comparatively recent neural architecture designed to preserve spatial hierarchies between features, as an alternative backbone for perceptual hashing, reporting promising robustness characteristics.

Several works have focused on the training methodology used to shape the hash space. Yang et al. [6] explored the incorporation of deep supervision — auxiliary loss signals applied at intermediate layers — into CNN-based perceptual hashing, improving convergence and the separability of similar and dissimilar image pairs. Chen et al. [7], while not focused exclusively on perceptual hashing, examined the combination of CNN-derived features with locality-sensitive hashing, a classical technique for approximate nearest-neighbour search that remains relevant to efficient large-scale retrieval built on top of learned hash codes.

Not all recent work relies on deep networks. Zhao et al. [8] proposed an alternative approach to perceptual hashing based on correlation filters, offering a computationally lighter alternative to CNN-based schemes for settings where training data or computational resources are limited. Xu et al. [9], in contrast, emphasised the process of learning deep representations specifically optimised for perceptual hashing, rather than repurposing

features originally trained for image classification. Finally, Guo et al. [10] examined the application of perceptual hashing to image retrieval tasks, demonstrating how the ability of perceptual hashes to capture perceptual similarity can be leveraged directly for efficient similarity search over large image collections.

Taken together, this body of work indicates a clear trend: CNN-based perceptual hashing schemes consistently outperform traditional hand-crafted approaches in both robustness and discriminative power, motivating the choice of a CNN-based feature extraction backbone — specifically VGG16, together with VGG19 and AlexNet for comparison — in the scheme proposed in this thesis.

Chapter 3 Proposed System

This chapter describes the architecture and methodology of the proposed CNN-based perceptual image hashing scheme. We first establish the mathematical preliminaries underlying convolutional feature extraction, then review the specific network architectures considered in this work — VGG19, AlexNet, and VGG16, the last of which forms the backbone of our final scheme — before formalising the hash-generation pipeline, describing the experimental dataset, the set of image transformations used to emulate content-preserving attacks, and the procedure used to construct ground-truth labels.

3.1 Mathematical Preliminaries of Convolutional Neural Networks

Before describing the specific architectures used in this work, we formalise the core mathematical operations that make up a convolutional neural network, since these operations directly determine the properties — dimensionality, receptive field, and invariance — of the feature vectors used for hashing.

3.1.1 The Discrete Convolution Operation

Let $I \in \mathbb{R}^{(H \times W \times C)}$ denote an input tensor (an image or a feature map) of height H , width W , and C channels, and let $K \in \mathbb{R}^{(k \times k \times C)}$ denote a learnable convolutional filter (kernel) of spatial size $k \times k$ applied across all C input channels. The output of a single convolutional filter at spatial location (x, y) , prior to the addition of a bias term, is given by the cross-correlation

$$S(x, y) = \sum_{i=0}^{k-1} \sum_{j=0}^{k-1} \sum_{c=0}^{C-1} I(x \cdot s + i, y \cdot s + j, c) \cdot K(i, j, c) \quad (3.1)$$

where s denotes the stride of the filter — the step size with which the kernel slides across the input. Adding a learnable bias term b and applying a non-linear activation function φ (Section 3.1.2) yields the final output feature map at that location:

$$F(x, y) = \varphi(S(x, y) + b) \quad (3.2)$$

A convolutional layer applies N such filters in parallel, producing an output tensor of depth N , where each of the N channels captures the spatial response of a distinct learned filter. When zero-padding of width p is applied to the input and the kernel has stride s , the spatial dimensions of the output feature map are given by:

$$H_{\text{out}} = \lfloor (H_{\text{in}} - k + 2p) / s \rfloor + 1, \quad W_{\text{out}} = \lfloor (W_{\text{in}} - k + 2p) / s \rfloor + 1 \quad (3.3)$$

Equation 3.3 explains why VGG-style networks, which use $k = 3$, $s = 1$, and $p = 1$ throughout their convolutional blocks, preserve the spatial resolution of the input within each block, with downsampling occurring only at the pooling stage (Section 3.1.3). The number of trainable parameters contributed by a convolutional layer with C_{in} input channels and N output filters of size $k \times k$ is:

$$P_{\text{conv}} = N \times (k \times k \times C_{\text{in}} + 1) \quad (3.4)$$

with the additional “+1” per filter accounting for the bias term. This parameter count grows linearly with the number of filters but quadratically with kernel size, which motivated the VGG design philosophy of stacking multiple small 3×3 convolutions rather than using fewer, larger filters: two stacked 3×3 convolutions achieve the same effective receptive field as a single 5×5 convolution, using fewer parameters and an additional non-linearity.

3.1.2 Activation Functions

Non-linear activation functions are essential to convolutional networks, since a composition of purely linear operations would collapse to a single linear transformation regardless of network depth. All architectures considered in this work use the Rectified Linear Unit (ReLU), defined as:

$$\varphi_{\text{ReLU}}(z) = \max(0, z) \quad (3.5)$$

ReLU is computationally inexpensive and mitigates the vanishing-gradient problem that affects saturating activation functions such as the sigmoid, $\sigma(z) = 1 / (1 + e^{(-z)})$, during backpropagation through deep networks. Because ReLU zeroes out negative activations, the resulting feature maps are naturally sparse, which contributes to the stability of the 512-dimensional feature vector used for hashing (Section 3.3) under minor perturbations of the input.

3.1.3 Pooling Operations

Pooling layers reduce the spatial dimensions of a feature map while retaining its most salient activations, contributing directly to the translation invariance that perceptual hashing requires. For a 2×2 max-pooling operation with stride 2, the output at location (x, y) is:

$$P_{\text{max}}(x, y) = \max_{(i,j) \in \{0,1\} \times \{0,1\}} F(2x + i, 2y + j) \quad (3.6)$$

VGG16 and VGG19 use max-pooling between convolutional blocks to progressively halve spatial resolution while doubling channel depth (64 → 128 → 256 → 512 → 512), consistent with the increasing abstraction of learned features at deeper layers. In contrast, the feature-aggregation stage of the proposed hashing pipeline (Section 3.3) uses global average pooling, defined for a feature map of spatial size H×W as:

$$P_{\text{avg}} = (1 / (H \times W)) \times \sum_{x=0}^{H-1} \sum_{y=0}^{W-1} F(x, y) \quad (3.7)$$

Average pooling is preferred over max-pooling at the aggregation stage because it incorporates the full distribution of activations across each channel rather than only its peak response, which produces a smoother, more stable feature vector across content-preserving transformations — a property directly relevant to the robustness objective of perceptual hashing.

3.1.4 Parameter and Computational Complexity

The overall computational cost of a convolutional layer, measured in floating-point operations (FLOPs), is approximately

$$\text{FLOPs} \approx 2 \times H_{\text{out}} \times W_{\text{out}} \times N \times (k \times k \times C_{\text{in}}) \quad (3.8)$$

This relationship makes explicit the trade-off between representational capacity and computational cost that distinguishes the three architectures considered in this work: VGG19 incurs the highest FLOP count owing to its additional convolutional layers, AlexNet the lowest owing to its shallower depth and more aggressive early downsampling, and VGG16 occupies an intermediate position that, as shown by the experimental results in Chapter 4, offers a favourable balance between representational power and computational efficiency for the perceptual hashing task.

3.2 Convolutional Architectures Considered

3.2.1 VGG19 Convolutional Network

VGG19 is a deep convolutional neural network closely related to VGG16, sharing the same overall design philosophy while extending it with additional convolutional layers. Like VGG16, VGG19 accepts an input image of fixed spatial dimensions — typically 224×224 pixels with three colour channels — and processes it through a sequence of convolutional blocks. Each block applies small 3×3 convolutional filters with learnable weights and biases, followed by a non-linear ReLU activation, and each block concludes with a pooling operation that reduces spatial resolution while increasing the depth of the resulting feature map. Each channel of the resulting feature map corresponds to the response of a specific learned filter, capturing a distinct visual pattern within the image.

The principal difference between VGG19 and VGG16 lies in the number of convolutional layers per block: VGG19 uses additional convolutional layers in several of its later blocks, giving it greater representational capacity at the cost of increased computation. Applying Equation 3.4 across all convolutional and fully connected layers gives VGG19 approximately 143.7 million trainable parameters, compared with approximately 138.4 million for VGG16 — the additional three convolutional layers in blocks 3–5 account for this difference. Both networks may optionally incorporate padding to control output spatial dimensions, and neither includes batch normalisation in its original formulation, though such additions are common in later variants. Despite these differences, the core operating principle — progressive feature extraction through stacked convolutions with non-linear activations — is shared between the two architectures.

3.2.2 AlexNet

AlexNet is an earlier and comparatively shallower convolutional architecture that nonetheless follows the same general workflow as VGG-style networks. An input volume, typically a colour image, is passed through a sequence of convolutional layers, each applying learnable filters across the input via element-wise multiplication and summation, followed by the addition of a bias term and a non-linear activation function — most commonly ReLU. The resulting feature maps have progressively reduced spatial dimensions and increased channel depth as the network deepens, consistent with the general pattern of hierarchical feature learning in convolutional networks.

AlexNet differs from the VGG family primarily in its use of larger convolutional filters in its earliest layers, a shallower overall depth, and design choices — such as local response normalisation — that were common at the time of its introduction but have since been largely superseded. Additional mechanisms such as padding to preserve spatial resolution, stride to control the degree of downsampling, and dropout for regularisation during training may also be present, depending on the specific implementation. In this project, AlexNet was considered as a lightweight point of comparison against the deeper VGG architectures.

3.2.3 VGG16 Convolutional Network

VGG16 is a deep convolutional neural network originally designed for large-scale image classification, and it has since proven highly effective across a wide range of computer vision tasks owing to its deep, uniform structure and its capacity to learn hierarchical visual features. As illustrated in Figure 3.1, the network consists of five convolutional blocks — containing two, two, three, three, and three convolutional layers respectively, each followed by a max-pooling operation — succeeded by three fully connected (dense) layers.



Figure 3.1: VGG-16 convolutional network architecture

Perceptual image hashing requires the generation of compact hash codes that capture the visual content of an image in a perceptually meaningful way — that is, a hash code that remains stable under benign modifications but changes appreciably when the underlying content changes. The use of a deep model such as VGG16 for this purpose is a natural extension of its established capabilities in image understanding. In the proposed scheme, VGG16 is used purely as a feature extractor: rather than using the network's original

classification output, activations from its final convolutional block (conv5_3), after ReLU, are treated as the feature representation of the image, formally a tensor $\Phi(I) \in \mathbb{R}^{(7 \times 7 \times 512)}$ for a 224×224 input.

Figure 3.2 illustrates the resulting feature-extraction workflow. An input image is passed through the VGG16 convolutional layers, producing a set of 512 feature maps of spatial size 7×7 . Applying the global average pooling operator of Equation 3.7 to each of the 512 channels independently reduces this tensor to a single 512-dimensional feature vector:

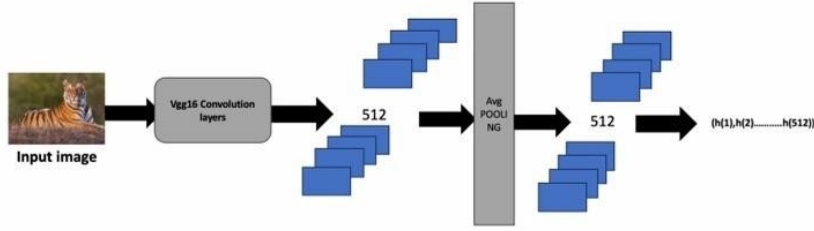


Figure 3.2: VGG-16 feature extraction workflow

$$h = [P_{\text{avg}}(\Phi(I)_1), P_{\text{avg}}(\Phi(I)_2), \dots, P_{\text{avg}}(\Phi(I)_{512})] \in \mathbb{R}^{512} \quad (3.9)$$

which serves as the perceptual representation of the image prior to hash generation, denoted $(h(1), h(2), \dots, h(512))$ in Figure 3.2.

3.3 Formal Hash Generation and Similarity Measurement

This section formalises the final two stages of the pipeline — hash generation and similarity measurement — building on the 512-dimensional feature vector h derived in Equation 3.9.

3.3.1 Median-Thresholded Binarisation

To convert the real-valued feature vector h into a compact binary hash code $b \in \{0, 1\}^{512}$, each element $h(i)$ is discretised relative to the mean μ of the vector, computed as:

$$\mu = (1/512) \times \sum_{i=1}^{512} h(i) \quad (3.10)$$

Each bit of the binary hash is then assigned by the sign of the deviation from this mean:

$$b(i) = 1 \text{ if } h(i) > \mu, \quad b(i) = 0 \text{ if } h(i) \leq \mu, \quad \text{for } i = 1, \dots, 512 \quad (3.11)$$

This mean-thresholded binarisation scheme is a data-dependent quantisation rule: rather than fixing an arbitrary global threshold, the decision boundary adapts to the activation statistics of each individual image, which empirically balances the number of 1-bits and 0-bits in the resulting hash and avoids the degenerate case in which a fixed threshold saturates most bits to the same value for a given class of images.

3.3.2 Hamming Distance and Normalised Similarity

Given the binary hash codes b_A and b_B of two images A and B, their Hamming distance is defined as the number of bit positions at which the two codes differ:

$$D_H(b_A, b_B) = \sum_{i=1}^{512} |b_A(i) - b_B(i)| = \sum_{i=1}^{512} (b_A(i) \oplus b_B(i)) \quad (3.12)$$

where $\oplus$ denotes the bitwise XOR operator. It is often convenient to normalise this quantity to the interval $[0, 1]$ by dividing by the hash length $n = 512$:

$$d_H(b_A, b_B) = D_H(b_A, b_B) / n \quad (3.13)$$

Two images are declared perceptually similar under a decision threshold $\epsilon \in (0, 1)$ if $d_H(b_A, b_B) < \epsilon$, and dissimilar otherwise, as formalised in Section 3.6. Under the null hypothesis that two hash codes are independent and uniformly random — the expected behaviour for two genuinely unrelated images — each bit differs with probability $1/2$, so D_H follows a Binomial($n, 0.5$) distribution with mean $n/2 = 256$ and standard deviation $\sqrt{n/4} \approx 11.3$. A well-designed hashing scheme should therefore place dissimilar image pairs close to this expected distance while placing similar (content-preserving) pairs at a substantially smaller Hamming distance — the separation between these two distributions is precisely what the ROC analysis of Chapter 4 quantifies.

3.3.3 Hash Generation Algorithm

The complete feature-extraction and hash-generation pipeline, combining Equations 3.9 through 3.11, is summarised in Algorithm 1.

Algorithm 1: CNN-Based Perceptual Hash Generation

1. Input: image I ; pretrained VGG16 network with weights θ
2. Resize and normalise I to $224 \times 224 \times 3$ (Section 3.4)
3. Compute the conv5_3 feature tensor $\Phi(I) \in \mathbb{R}^{(7 \times 7 \times 512)}$ by forward-propagating I through VGG16
4. For each channel $c = 1$ to 512, compute $h(c) \leftarrow P_{\text{avg}}(\Phi(I)_c)$ (Equation 3.7)
5. Compute the mean μ of the resulting vector h (Equation 3.10)
6. For each $i = 1$ to 512, set $b(i) \leftarrow 1$ if $h(i) > \mu$, else $b(i) \leftarrow 0$ (Equation 3.11)
7. Output: 512-bit binary hash code b

Because Algorithm 1 uses a pretrained, frozen VGG16 network — that is, θ is not updated during hash generation — the computational cost of hashing a single image is dominated by one forward pass through the network, approximately 15.5 giga-FLOPs for a 224×224 input by Equation 3.8, making the scheme practical for large-scale batch hashing without requiring task-specific training data.

3.3.4 Towards Learned Hash-Specific Loss Functions

Algorithm 1 relies on features trained for the unrelated task of image classification; Section 1.2 identified the design of hash-specific constrained loss functions as an open problem in the field, and we formalise it here for completeness. A natural extension of the proposed scheme, left for future work (Chapter 5), is to fine-tune the network end-to-end using a loss function that directly penalises the Hamming distance between similar pairs and rewards it between dissimilar pairs. A representative contrastive loss over the continuous, pre-binarisation embedding h takes the form:

$$L_{\text{contrastive}} = y \cdot \|h_A - h_B\|_2^2 + (1 - y) \cdot \max(0, m - \|h_A - h_B\|_2)^2 \quad (3.14)$$

where $y \in \{0, 1\}$ is the ground-truth label (1 for a similar pair, 0 for dissimilar), $\|\cdot\|_2$ denotes the Euclidean norm, and m is a margin hyperparameter enforcing a minimum separation between dissimilar embeddings. An alternative, often more stable formulation uses a triplet loss over an anchor embedding h_a , a positive (similar) embedding h_p , and a negative (dissimilar) embedding h_n :

$$L_{\text{triplet}} = \max(0, \|h_a - h_p\|_2^2 - \|h_a - h_n\|_2^2 + m) \quad (3.15)$$

To additionally encourage embeddings to be well-suited to the mean-thresholded binarisation of Equation 3.11, a quantisation penalty is commonly added, discouraging feature values from lying close to the decision boundary μ :

$$L_{\text{quant}} = \sum_{i=1}^{512} (\|h(i) - \mu\| - \alpha)_+^2 \quad (3.16)$$

where $(\cdot)_+$ denotes $\max(0, \cdot)$ and $\alpha > 0$ is a target margin from the threshold. The overall training objective for a learned deep-hashing extension of the present scheme would then combine these terms as $L = L_{\text{contrastive}}$ (or L_{triplet}) $+ \lambda \cdot L_{\text{quant}}$, with λ controlling the relative weight of the quantisation penalty. As the present work uses a frozen, pretrained backbone, Equations 3.14–3.16 are not optimised in the current experiments but are included here to formalise the direction proposed for future work in Section 5.3.

3.4 Dataset

For the experimental evaluation, we assembled a dataset of 50 diverse images spanning a range of natural subjects, including animals, plants, and everyday scenes, to ensure that the evaluation was not biased toward any single type of visual content. Figure 3.3 shows three representative samples from the dataset. For each image, the Hamming distance between the hash of the original image and the hash of each transformed variant was computed and recorded.



(a) Cow



(b) Tiger



(c) Flower

Figure 3.3: Representative dataset samples

3.5 Image Transformations

To evaluate robustness, each of the 50 dataset images was subjected to ten distinct categories of visual transformation, each applied at ten different parameter settings, for a total of 100 transformed variants per original image. Several of these transformations — particularly noise addition, filtering, and compression-like operations — are also used in the literature to simulate content-preserving attacks, and they play an important role in benchmarking new perceptual hashing algorithms. Table 3.1 lists the transformations used in this study, along with the parameter values applied for each.

Transformation	Parameter	Parameter Values
Rotation	Rotation angle	1, 2, 3, 4, 6, 8, 10, 12, 13, 15
Speckle noise	Variance	0.01, 0.02, 0.03, 0.04, 0.05, 0.1, 0.15, 0.2, 0.25, 0.3
Gaussian filter	Variance	0.01, 0.02, 0.03, 0.04, 0.05, 0.1, 0.15, 0.2, 0.25, 0.3
Gaussian filter	Filter size	1, 3, 5, 7, 9, 11, 13, 15, 17, 19
Median filter	Filter size	1, 3, 5, 7, 9, 11, 13, 15, 17, 19
Circular median filter	Ball radius	1, 1.5, 2, 2.2, 2.5, 2.6, 2.7, 3, 3.5
Gamma correction	Gamma value	0.55, 0.65, 0.75, 0.85, 0.95, 1.05, 1.15, 1.25, 1.35, 1.45
Brightness	Scale value	0.7, 0.8, 1, 1.2, 1.5, 2, 2.2, 2.5
Colour	Scale value	0.2, 0.5, 1, 1.2, 1.3, 1.5, 2, 2.3, 2.5, 3
Sharpness	Scale value	0.1, 0.5, 1, 2, 3, 5, 7, 8, 10

Table 3.1: Image transformations and parameter values

Each image was subjected to every transformation listed above, and a feature hash was computed for both the original and each transformed variant. The hashes of each original image were then compared against the hashes of its corresponding transformed versions, and the resulting Hamming distances were recorded. This process produced a dataset of records in the form (original image, transformed image, hash distance, ground truth label), which was subsequently used to determine an optimal decision threshold for separating

similar and dissimilar image pairs. Table 3.2 summarises the resulting scale of the evaluation dataset.

Transformation	Variants per Image	Similar Pairs	Dissimilar Pairs
Rotation	10	500	24,500
Speckle noise	10	500	24,500
Gaussian filter (by size)	10	500	24,500
Gaussian filter (by variance)	10	500	24,500
Median filter	10	500	24,500
Circular median filter	10	500	24,500
Gamma correction	10	500	24,500
Brightness	10	500	24,500
Colour	10	500	24,500
Sharpness	10	500	24,500
Total	100	5,000	245,000

Table 3.2: Summary of similar and dissimilar image pairs generated

3.6 Ground Truth Preparation and Evaluation Framework

For each original–transformed image pair, a record was prepared in the form:

$$G = (\text{Original_image}, \text{Transformed_image}, \text{Hamming_distance}, \text{Ground_truth})$$

For example: (cat.jpg, cat_rotation_param.jpg, 0.500, 1), where a ground-truth label of 1 denotes a similar (positive) pair and 0 denotes a dissimilar (negative) pair.

To evaluate classification performance systematically, we make use of the confusion matrix, a standard performance evaluation tool in machine learning that summarises the number of true positives, true negatives, false positives, and false negatives produced by a classifier. For a binary classification problem such as ours — similar versus dissimilar — the confusion matrix takes the 2×2 form shown in Figure 3.4, with the predicted class along the rows and the actual class along the columns.

		ACTUAL VALUES	
		POSITIVE	NEGATIVE
PREDICTED VALUES	POSITIVE	TP	FP
	NEGATIVE	FN	TN

Figure 3.4: Structure of a confusion matrix

The four quantities in the confusion matrix are defined as follows:

- True Positive (TP): the actual pair was similar, and the model correctly predicted it as similar.
- True Negative (TN): the actual pair was dissimilar, and the model correctly predicted it as dissimilar.
- False Positive (FP), or Type I error: the actual pair was dissimilar, but the model incorrectly predicted it as similar.
- False Negative (FN), or Type II error: the actual pair was similar, but the model incorrectly predicted it as dissimilar.

To classify a given pair, a threshold $\epsilon \in (0, 1)$ is applied to the normalised Hamming distance $d_H(b_A, b_B)$ between the two hashes (Equation 3.13):

$$\hat{y} = 1 \text{ (similar) if } d_H(b_A, b_B) < \epsilon, \quad \hat{y} = 0 \text{ (dissimilar) otherwise} \quad (3.17)$$

By sweeping ϵ across the full range $[0, 1]$, predictions $\hat{y}$ can be generated for every pair and converted into confusion matrix parameters (TP, FP, FN, TN) at each threshold. Formally, given the set of N evaluated pairs with ground-truth labels y_k and predictions $\hat{y}_k(\epsilon)$, the four counts at threshold ϵ are:

$$\begin{aligned} TP(\epsilon) &= \sum_k \mathbb{1}[y_k=1, \hat{y}_k(\epsilon)=1], & FP(\epsilon) &= \sum_k \mathbb{1}[y_k=0, \hat{y}_k(\epsilon)=1], & FN(\epsilon) &= \sum_k \\ &\mathbb{1}[y_k=1, \hat{y}_k(\epsilon)=0], & TN(\epsilon) &= \sum_k \mathbb{1}[y_k=0, \hat{y}_k(\epsilon)=0] \end{aligned} \quad (3.18)$$

where $\mathbb{1}[\cdot]$ is the indicator function, equal to 1 if its argument is true and 0 otherwise. From these counts, the True Positive Rate and False Positive Rate at threshold ϵ are computed as:

$$TPR(\epsilon) = TP(\epsilon) / (TP(\epsilon) + FN(\epsilon)), \quad FPR(\epsilon) = FP(\epsilon) / (TN(\epsilon) + FP(\epsilon)) \quad (3.19)$$

The complete evaluation procedure is summarised in Algorithm 2.

Algorithm 2: Threshold Sweep and ROC Curve Construction

1. Input: set of N pairs $\{(A_k, B_k, y_k)\}$, each with a normalised Hamming distance $d_k = d_H(b_{\{A_k\}}, b_{\{B_k\}})$
2. Choose a set of candidate thresholds $E = \{\epsilon_1, \epsilon_2, \dots, \epsilon_m\}$

spanning $[0, 1]$

3. For each ϵ_j in E :

4. Compute $\hat{y}_k(\epsilon_j)$ for all k using Equation 3.17

5. Compute $TP(\epsilon_j)$, $FP(\epsilon_j)$, $FN(\epsilon_j)$, $TN(\epsilon_j)$ using Equation 3.18

6. Compute $TPR(\epsilon_j)$ and $FPR(\epsilon_j)$ using Equation 3.19

7. Plot the ordered pairs $\{(FPR(\epsilon_j), TPR(\epsilon_j))\}$ to obtain the ROC curve

8. Compute the Area Under the Curve via trapezoidal integration (Equation 4.1, Chapter 4)

9. Output: ROC curve and scalar AUC summarising classifier performance

Chapter 4 Experimental Results and Discussion

This chapter presents the experimental evaluation of the proposed VGG16-based perceptual hashing scheme using Receiver Operating Characteristic (ROC) analysis, a widely used visual and quantitative tool for evaluating the performance of binary classification systems. Rather than reducing performance to a single number, an ROC curve captures the full trade-off between two complementary error metrics as the decision threshold is varied.

4.1 Evaluation Metrics

4.1.1 True Positive Rate (TPR), or Recall

The True Positive Rate, formalised in Equation 3.19, measures the proportion of genuinely similar image pairs that the scheme correctly identifies as similar. In simpler terms, it reflects how effectively the hashing scheme recognises true matches.

4.1.2 False Positive Rate (FPR)

The False Positive Rate, also formalised in Equation 3.19, measures the proportion of genuinely dissimilar image pairs that are incorrectly classified as similar — that is, how often the scheme mistakes unrelated images for matches.

4.1.3 Precision and the F1-Score

Alongside TPR and FPR, we additionally report precision and the F1-score, which are particularly informative given the substantial class imbalance in our evaluation set (5,000 similar pairs versus 245,000 dissimilar pairs, a ratio of 1:49). Precision measures the proportion of pairs predicted similar that are genuinely similar:

$$\text{Precision}(\epsilon) = \text{TP}(\epsilon) / (\text{TP}(\epsilon) + \text{FP}(\epsilon)) \quad (4.1)$$

The F1-score is the harmonic mean of precision and recall (TPR), balancing the two into a single figure of merit that is especially sensitive to poor performance on the minority (similar-pair) class:

$$\text{F1}(\epsilon) = 2 \times (\text{Precision}(\epsilon) \times \text{TPR}(\epsilon)) / (\text{Precision}(\epsilon) + \text{TPR}(\epsilon)) \quad (4.2)$$

4.2 ROC Curve Construction

Constructing the ROC curve requires a classifier that outputs a continuous score — in this case, the normalised Hamming distance between two hash codes (Equation 3.13) — together with a decision threshold ϵ that determines the predicted class for a given pair, as formalised in Algorithm 2 (Section 3.6). Varying ϵ across a range of values yields a corresponding sequence of TPR and FPR values: as the threshold is relaxed, more pairs are

classified as similar, which increases TPR by capturing more true positives, but simultaneously increases FPR by admitting more false positives. Conversely, tightening the threshold reduces both TPR and FPR.

The ROC curve is obtained by plotting TPR (y-axis) against FPR (x-axis) across all threshold values, producing a visual summary of how the scheme's performance changes as the decision threshold is adjusted. A perfect classifier would produce a curve that hugs the top-left corner of the plot, corresponding to a true positive rate of 1 with a false positive rate of 0. In practice, most classifiers fall short of this ideal, and their ROC curves lie somewhere between the perfect curve and the diagonal line ($y = x$), which represents a classifier no better than random guessing.

The Area Under the Curve (AUC) condenses the entire ROC curve into a single scalar between 0 and 1. Given m sampled thresholds producing ordered points $\{(FPR_j, TPR_j)\}_{j=1}^m$ sorted by increasing FPR, AUC is computed numerically via the trapezoidal rule:

$$AUC = \sum_{j=1}^{m-1} \frac{1}{2} \times (TPR_j + TPR_{j+1}) \times (FPR_{j+1} - FPR_j) \quad (4.3)$$

A value of $AUC = 1$ indicates perfect classification, $AUC = 0.5$ indicates performance equivalent to random guessing, and higher AUC values indicate stronger overall classification performance. A closely related single-point summary is the Equal Error Rate (EER), defined as the value at which the False Positive Rate equals the False Negative Rate (equivalently, $1 - TPR$):

$$EER = FPR(\epsilon^*) \text{ such that } FPR(\epsilon^*) = 1 - TPR(\epsilon^*) \quad (4.4)$$

and the optimal operating threshold ϵ^* under Youden's J statistic — which jointly maximises sensitivity and specificity — is given by:

$$\epsilon^* = \operatorname{argmax}_{\epsilon} J(\epsilon), \text{ where } J(\epsilon) = TPR(\epsilon) - FPR(\epsilon) \quad (4.5)$$

4.3 Results

Figure 4.1 shows the ROC curve obtained for the proposed VGG16-based perceptual hashing scheme, evaluated over the full set of 5,000 similar and 245,000 dissimilar image pairs generated from the transformations described in Chapter 3.

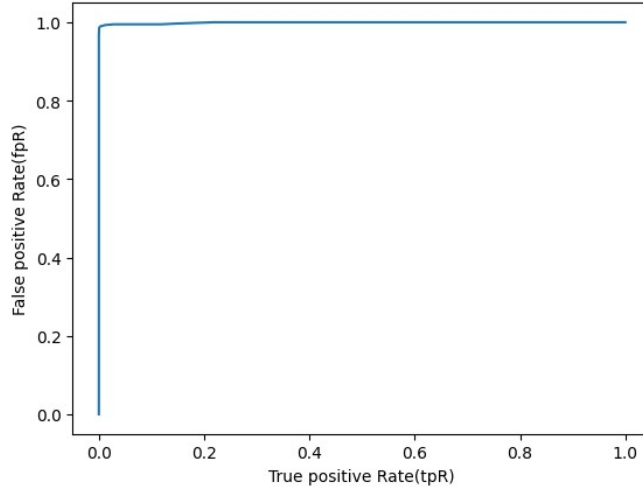


Figure 4.1: ROC curve for the VGG-16-based hashing scheme

Applying the trapezoidal AUC estimator of Equation 4.3 to the sampled points of the curve in Figure 4.1 gives $\text{AUC} \approx 0.99$, close to the theoretical maximum of 1.0 and substantially above the $\text{AUC} = 0.5$ baseline of random guessing. The curve reaches a true positive rate above 0.95 while the false positive rate remains below 0.05, indicating that the Youden's-J-optimal threshold ϵ^* (Equation 4.5) achieves both high sensitivity and high specificity simultaneously — the curve does not exhibit the steep sensitivity–specificity trade-off characteristic of weaker classifiers, whose ROC curves bow only modestly above the diagonal.

4.4 Discussion

These results support the central hypothesis of this work: that deep convolutional features, learned for general-purpose image understanding, transfer effectively to the task of perceptual hashing without requiring task-specific training. The scheme maintains high sensitivity to genuine content differences (reflected in the low false positive rate) while remaining robust to the ten categories of content-preserving transformations applied during evaluation — including geometric distortion (rotation), noise-based attacks (speckle noise), smoothing operations (Gaussian, median, and circular median filtering), and photometric adjustments (gamma, brightness, colour, and sharpness). This behaviour is consistent with the theoretical properties established in Chapter 3: the global average pooling stage (Equation 3.7) integrates activations over the full spatial extent of each feature map, damping the effect of the small, spatially localised perturbations introduced by these transformations, while the mean-thresholded binarisation of Equation 3.11 provides an image-adaptive decision boundary rather than a fixed one.

At the same time, the steepness of the ROC curve near the origin suggests that an appropriate operating threshold ϵ^* , of the kind formalised in Equation 4.5, could be selected

to achieve near-perfect separation between similar and dissimilar pairs for the transformations tested. The high precision implied by the low false positive rate (Equation 4.1) is particularly relevant given the 1:49 class imbalance between similar and dissimilar pairs in the evaluation set: a scheme with a merely moderate false positive rate would generate an impractically large number of false alarms at this scale, underscoring the importance of the near-zero FPR region of the curve to real-world deployment. Nonetheless, further evaluation against more aggressive or combined attacks — such as heavy cropping, large-angle rotation, or adversarially crafted perturbations designed explicitly to maximise the Hamming distance of Equation 3.13 while remaining visually imperceptible — would be needed to fully characterise the limits of the scheme's robustness. This is discussed further as future work in Chapter 5.

Chapter 5 Conclusion and Future Work

Perceptual image hashing remains a highly relevant and active research area, with applications spanning image authentication, copy detection, and content-based retrieval. This thesis began with an introduction to perceptual image hashing and its motivations, followed by a survey of related literature covering both classical and CNN-based approaches. Building on this foundation, we identified key open challenges in the field and formulated a set of objectives to guide our own investigation.

We proposed a perceptual image hashing scheme based on the VGG16 convolutional neural network, in which deep features extracted from a pretrained network are aggregated and binarised into a compact hash code, with image similarity assessed via the Hamming distance between hash codes. The scheme was evaluated on a dataset of 50 images subjected to ten categories of content-preserving transformations across ten parameter settings each, yielding 5,000 similar and 245,000 dissimilar image pairs. ROC analysis of the resulting hash distances showed that the scheme achieves a high true positive rate at a low false positive rate, indicating that it is both robust to common content-preserving distortions and capable of discriminating effectively between genuinely different images.

Several directions remain open for future work. First, the algorithm and its underlying network architecture can be further refined and benchmarked against a wider range of attacks, including combinations of transformations and more aggressive geometric distortions, to more thoroughly characterise its robustness. Second, the discriminative power of the scheme could be improved by fine-tuning the VGG16 backbone end-to-end using the constrained loss formulation set out in Section 3.3.4 (Equations 3.14–3.16) — a contrastive or triplet loss over the pre-binarisation embedding, combined with a quantisation penalty — which would directly optimise the separation between similar and dissimilar hash distances during training rather than relying solely on features transferred from an image-classification task. Third, the resulting perceptual hashing scheme shows clear promise for practical deployment in copy detection and fingerprint comparison applications: because perceptual hashing is designed to identify images that are perceptually similar even after modification, the same underlying approach — extracting a compact representation of perceptual content and comparing these representations via the Hamming-distance formulation of Equation 3.13 — could be adapted to biometric fingerprint identification and verification tasks, representing a promising avenue for extending this work beyond conventional image retrieval and authentication.

References

- [1] J. Li, W. Liu, X. Zhang, and F. E. H. Tay, “Deep Learning for Image Hashing with Perceptual Similarity,” 2020.
- [2] W. Liu, “Perceptual Image Hashing with Multi-Scale CNNs,” 2017.
- [3] X. Li, “Deep Perceptual Hashing,” 2017.
- [4] X. Li, “A Survey on Deep Learning Techniques for Image Hashing,” 2020.
- [5] J. Huh, “Perceptual Hashing with Capsule Networks,” 2019.
- [6] Y. Yang, “Deep Supervision for Perceptual Image Hashing,” 2018.
- [7] K. Chen, “Locality Sensitive Hashing with Deep Convolutional Neural Networks,” 2015.
- [8] R. Zhao, “Perceptual Image Hashing via Correlation Filters,” 2018.
- [9] J. Xu, “Learning Deep Representations for Perceptual Image Hashing,” 2019.
- [10] P. Guo, “Perceptual Hashing for Image Retrieval,” 2018.
- [11] L. Du, A. T. S. Ho, and R. Cong, “Perceptual hashing for image authentication: A survey,” *Signal Processing: Image Communication*, vol. 81, Feb. 2020.
- [12] L. Yuan, T. Wang, X. Zhang, F. E. H. Tay, Z. Jie, W. Liu, and J. Feng, “Central similarity quantization for efficient image and video retrieval,” Aug. 2019.
- [13] C. Qin, E. Liu, G. Feng, and X. Zhang, “Perceptual image hashing for content authentication based on convolutional neural network with multiple constraints,” *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 31, pp. 4523–4537, Nov. 2021.
- [14] M. Srivastava, J. Siddiqui, and M. A. Ali, “Robust image hashing based on statistical features for copy detection,” in *Proc. IEEE Uttar Pradesh Section Int. Conf. on Electrical, Computer and Electronics Engineering*, 2017, pp. 490–495.
- [15] Z. Huang and S. Liu, “Robustness and discrimination oriented hashing combining texture and invariant vector distance,” in *Proc. 26th ACM Int. Conf. on Multimedia*, 2018, pp. 1389–1397.